# MODULE - 3

Introduction to MongoDB:
What is MongoDB, Why MongoDB, Terms used in RDBMS & MongoDB, Data Types in MongoDB, MongoDB Query language.
TB1: Ch-6: 6.1 - 6.5

## 6.1 WHAT IS MONGODB?

It is,

1. Cross-platform
2. Open Source
3. Non-relational
4. Distributed
5. NoSQL
6. Document-oriented data ~~Source~~ store.

## 6.2 WHY MONGODB?

### 6.2.1 Using Java Script Object Notation (JSON)

→ JSON is extremely expressive
→ MongoDB actually does not use JSON but BSON
→ It is used to store Complex data structures

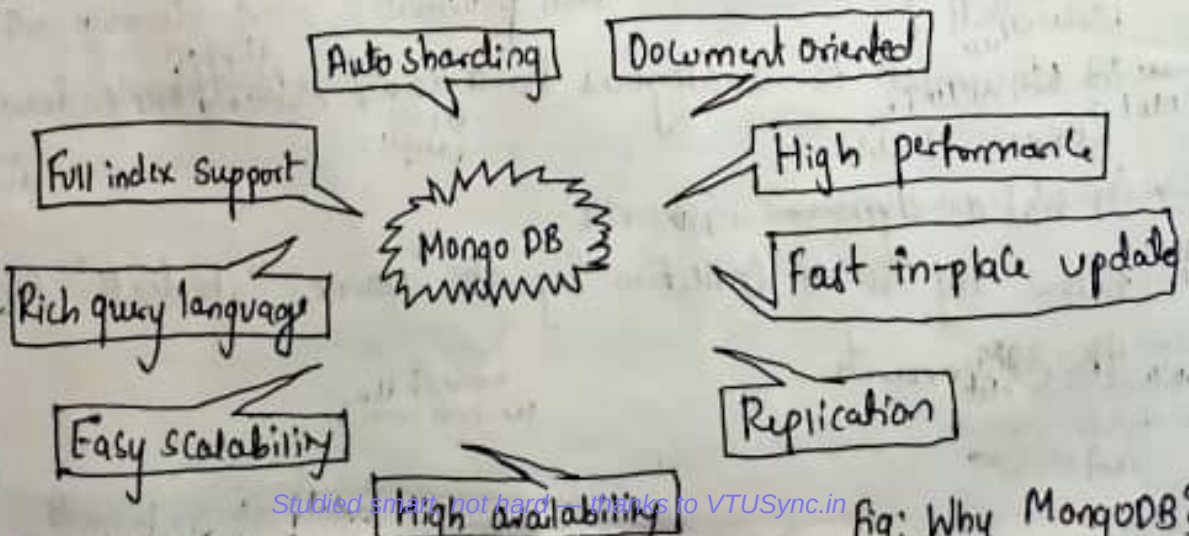



fig: Why MongoDB?

## 6.2.2 Creating or Generating a Unique Key

→ Each JSON have a unique identifier.
→ It is the _id key
→ It is similar to the primary key in relational databases.

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Timestamp | | | | Machine ID | | | Process ID | | Counter | | |

Database:
→ It is a collection of collections.
→ It is like a container for collections

Collection:
→ It is analogous to a table of RDBMS
→ It is created on demand
→ It gets created the first time that you attempt to save a document that references it.
→ A collection exists within a single database.
→ It holds several MongoDB documents.
→ It does not enforce a schema.

Document:
→ A document is analogous to a row/record/tuple in an RDBMS table.
→ It has a dynamic schema.

Below fig is a collection by the name "students" containing three documents

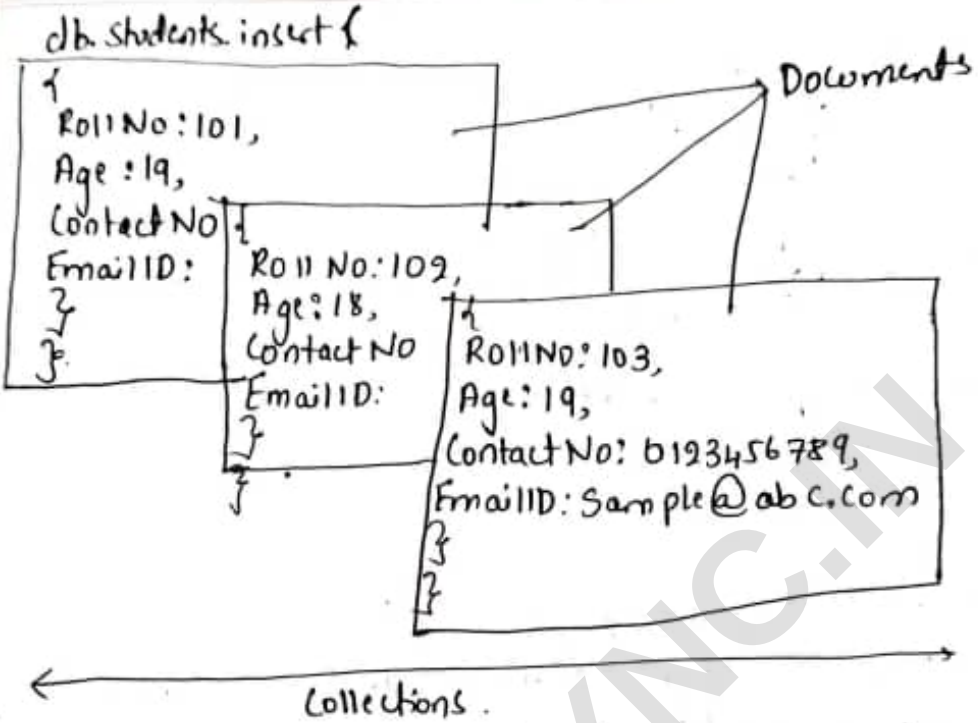



Fig: A Collection "students" containing 3 documents.

## Replication:

→ Replication provides data redundancy & high availability.
→ It helps to recover from h/w failure & service interruptions.
→ In MongoDB, the replica set has a single primary & several secondaries.
→ The primary logs all write requests into its oloperation logs.
→ The Oplog is then used by the Secondary replica members to synchronize their data.
→ In the below diagram. The client usualy read from the primary
→ However, the client can also specify, a read preference that will then direct the read operations to the secondary.

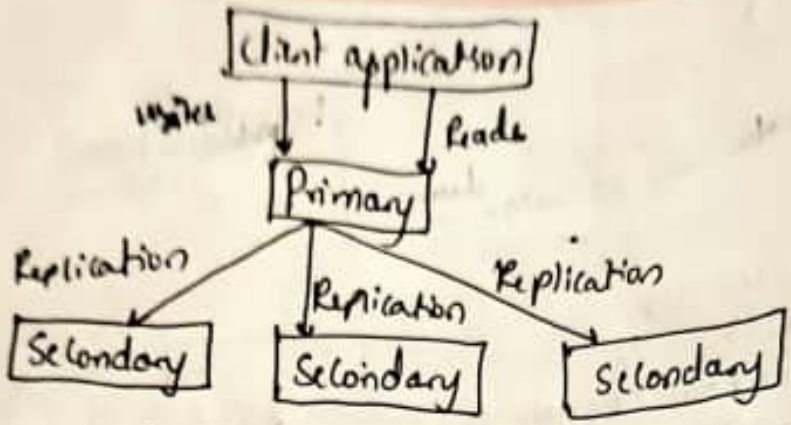



Fig: The process of REPLICATION in MongoDB

## Sharding:

→ It is akin to horizontal scaling
→ It means that the large dataset is divided & distributed over multiple servers or shards
→ Each shard is independent database & collectively they would constitute a logical database

The prime advantages of sharding

1) It reduces the amount of data that each shard needs to store & manage.
2) It reduces the no. of operations that each shard handles
Ex: If we insert data, the appln needs to access only that shard which houses that data.

## 6.3 TERMS USED IN RDBMS AND MONGODB

| RDBMS | MongoDB |
|---|---|
| Database | Database |
| Table | Collection |
| Record | Document |
| Columns | Fields / Key value pairs |
| Index | Index |

| RDBMS | MongoDB |
|---|---|
| Joins | Embedded documents |
| Primary key | Primary key (_id is a identifier) |

| | MySQL | Oracle | MongoDB |
|---|---|---|---|
| Database Server | Mysqld | Oracle | Mongod |
| Database client | Mysql | SQL Plus | mongo |

6.3.1 Create Database

Syntax:

```
use DATABASE_Name
```

To create a database by the name "myDB" the syntax is

Syntax:
```
use myDB
```

To confirm the existence of your database,

Syntax:
```
db
```

```
> db;
myDB;
>
```

To get the list of all databases,

```
show dbs;
```

```
>show dbs;
admin (empty)
local 0.07 8GB
test 0.078GB
```

Notice that newly created database "myDB" does not show up in the list. The reason is it needs to have at least one document to show up in the list.

6.3 Drop Database

Syntax: `db.dropDatabase();`

→ To drop the newly created "myDB", first ensure you are currently placed in "myDB" database & then use the db.dropDatabase()

```
use myDB;
db.dropDatabase();
```

→ If no database is selected, the default database "test" is d

6.4 DATA TYPES IN MONGODB:

| | |
|---|---|
| String | Most commonly used data type |
| Integer | Can be 32-bit or 64-bit (depends on the server). |
| Boolean | To store a true/false value. |
| Double | To store a floating point (real values) |
| Min/Max keys | To compare a value against the lowest or highest BSON elements |
| Arrays | To store arrays or list or multiple values into one key. |
| Timestamp | To record when a document has been modified or added. |
| NULL | To store a NULL value. Missing or unknown value. |
| Date | To store the current date or time in Unix time format |
| Object ID | To store the document's id. |
| Binary data | To store the binary data (images, binaries etc) |

Code — To store javascript code into the document

Regular expression — To store regular expression

A few Commands worth looking at are as follows

1) To report the name of the current database:

```
>db
test
>
```

2) To display the list of databases:

```
>show dbs
admin (empty)
local 0.078GB
myDB1 0.078GB
>
```

3) To switch to a new database, for ex, myDB1:

```
>use myDB1
Switched to db myDB1
>
```

4) To display the list of collections in the current database

```
>show collections
System.indexes
System.js
>
```

5) To display the current version of the MongoDB server.

```
>db.version()
2.6.1
>
```

To display the statics that reflect the use state of a database:

```
>db.stats()
{
    "db"  : "myDB1"
    "collection" : 3,
    "objects" :6,
    "avgObjSize" : 122.6667,
    "dataSize": 736,
    "StorageSize" : 24576,
    "numExtents":3,
    "indexSize" : 8176,
    "fileSize" : 67108864,
    "nsSizeMB" : 16,
    "dataFileVersion" : {
        "major" : 4,
        "minor": 5
    },
    extentFreeList" : {
        "num" : 14,
        "totalSize": 974848
    },
    "ok": 1
}
```

To display
Type db.help() in MongoDB client to get a list of commands:

```
>db.help();
```

## 6.5 MongoDB Query Language:

CURD (Create, Read Update, & Delete) operations in Mongo

Create → Creation is done using insert() or update() or save() method

Read → Reading of data is performed using find() method
Update → Update to data is done using Update() method with UPSERT set to false
Delete → a document is deleted using remove() method

Various methods available in MongoDB Shell to deal with data in next few scenarios have been designed as follows:
Objective: What is it that we are trying to achieve here?
Input: What is i/p that has been given to ~~accomplish the task~~ us to act upon?
Act: The actual statement/command to accomplish the task at hand
Outcome: The result/o/p as a consequence of executing the statement

1. Objective: To create a collection by the name "person"
Before Creating will check the list,
> show collections  o/p: students
food
Act: The statement to create the collection is

```
db.createCollection("Person");
```

o/p: Person
students
food

2) Objective: To drop a collection by the name "food";
>show collections;
Act: Statement to drop the collection is

```
db.food.drop();
```

Outcome: ~~Person, cott~~ Person
Students

## 6.5.1 Insert method

Syntax:

```
db.students.insert(   <-- Collection
{
  RollNO: 101,        <-- Field: value
  Age: 19,            <-- Field: value
  ContactNO: 0123456789,  <-- Field: value
  EmailID: Sample@abc.com <-- Field: value
}
)
```

1. Objective: To create a collection by the name "students" & insert documents

I/p: Check the list of existing collections

```
>show collections
System.collectionsindexes
System.js
>
```

Act: Create a collection by the name "students" & the details in it

```
>db.students.insert({_id:1, StudName: "Michelle Jacintha", Grade: "VII",
                     Hobbies: "Internet Surfing"});
WriteResult({"nInserted": 1})
```

Outcome: check if the collection has been successfully created

```
>show collections
Students
System.indexes
System.js
>
```

Check if the document for Student "Michelle Jacintha" has been inserted

```
>db.students.find();
{"_id": 1, "StudName": "Michelle Jacintha", "Grade": "VII", "Hobbies":
>     "Internet Surfing"}
```

To format the result, one can add the pretty() method to the operation

```
>db.students.find().pretty();
{
  "_id":1,
  "StudName": "Michelle Jacintha",
  "Grade": "VII",
  "Hobbies": "Internet Surfing"
}
>
```

2) Objective: Insert another document into the Collection.

I/p: Check the documents in the "students" collection before proceeding.

```
>db.Students.find().pretty();
```

Act:

```
>db.Students.Insert({_id:2,StudName: "Mabel Mathews", Grade: "VII",
    Hobbies: "Baseball"});
WriteResult({ "nInserted" : 1})
>
```

Outcome:

```
>db.Students.find().pretty();
{
  "_id" :11,
  "StudName": "Michelle Jacintha",
  "Grade": "VII",
  "Hobbies": "Internet Surfing"
}
{
  "_id":2,
  "StudName": "Mabel Mathews",
  "Grade": "VII",
  "Hobbies": "Baseball"
}
```

3. Objective : Insert the document for "Aryan David" into the Students Collection only if it does not already exist in the collection. However, if it is already present in the Collection, then update the document with new values. (Update his Hobbies from "Skating" to "Chess").
Use "Update else insert" [If there is an existing document, it will attempt to update it, if there is no existing document then it will insert it).

I/P: Check the documents in the "Students" Collection before proceding

```
>db.students.find().pretty();
```

Alt:

```
>db.students.update({_id:3, StudName: "Aryan David", Grade: "VII"},
{$set:{Hobbies: "Skating"}}, {upsert: true});
WriteResult({"nMatched":0, "nUpserted":1, "nModified":0, "_id":3})
>
```

Outcome: Confirm the presence of the document of "Aryan David" in the "Students" Collection.

```
>db.students.find({_id:3});
{"_id":3, "Grade":"VII", "StudName":"Aryan David", "Hobbies":"Skating
>
```

4. Objective : To demonstrate Save method to insert a document for student "Vansi Bapat" in the "Students" Collection.

I/P : Check the document in the "Students" Collection before proceddling

```
>db.students.find().pretty();
{
    "_id":1,
    "StudName": "Michella Jacintha",
    "Grade": "VII",
    "Hobbies": "Internet Surfing"
}
{ "_id":2,
  "StudName": "Mabel Mathews",
  "Grade": "VII",
  "Hobbies": "Baseball"
```

```
{ "_id" : 3,
  "Grade" : "VII",
  "StudName" : "Aryan David",
  "Hobbies" : "Chess" }
```

Act :

```
>db.Students.Save ({StudName : "Vamsi Bapat", Grade : "VI"})
WriteResult ({ "nInserted" : 1})
```

Outcome :

```
>db.students.find().pretty();
{
  "_id" : 1,
  "StudName" : "Michelle Jacintha",
  "Grade" : "VII",
  "Hobbies" : "Internet Surfing"
}
{ "_id" : 2,
  "StudName" : "Mabel Mathews",
  "Grade" : "VII",
  "Hobbies" : "Baseball"
}
{ "_id" : 3
  "Grade" : "VII",
  "StudName" : "Aryan David",:
  "Hobbies" : "Chess",
}
{ "_id" : ObjectId("546dd0ea7fba710999bb94d"),
  "StudName" : "Vamsi Bapat",
  "Grade" : "VI"
}
```

## 6.5.2 Save() Method :

→ The Save() method will insert a new document if the document with the specified _id does not exist

→ If it already exists, it replaces the existing document with new one.

1. Objective : Insert the document of "Hersch Gibbs" into the Students Collection using the Update method.

- First try with upsert set to false & the with upsert set to true

I/p : check the document in the "Students" collection before proceding.

```
>db.students.find();
{"_id":1, "StudName":"Michelle Jacintha","Grade":"VII", "Hobbies":
                                                    Internet Surfing"}
{"_id":2, "StudName":"Mabel Mathews", "  ":"VII", "Hobbies":ba
{"_id":3, "  ":"Aryan David", "Hobbies":"chess"}
{"_id":4, ObjectId("54c0d0e047fba710799bb94d"),"StudName":"Vansi
                                    Bapat", "Grade":"VI"}
```

Act: Update method with upsert set to false.

```
>db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade:"VII"},
    {$set:{Hobbies:"Graffiti"}},{upsert:false});
WriteResult({"nMatched":0, "nUpserted":0 ,"nModified":0})
```

- Update method with upsert set to true.

```
>db.Students.update({_id:4, StudName:"Hersch Gibbs", Grade:"VII"},
    {$set:{Hobbies:"Graffiti"}},{upsert:true});
WriteResult({"nMatched":0 ,"nUpserted":1 ,"nModified":0 , "_id":4})
```

nUpserted:1 implies that a document with _id:4 is inserted

Outcome : Confirm the presence of the document of "Hersch Gibbs" in the "Students" Collection.

6.5.3 Adding a new field to an Existing Document - Update Method :

```
db.Students.Updat (        ← Collection
{rng:{$gt 18}},            ← Updat Criteria
{$set:{Status:"A"}}        ← Updat Action
{multi:true}               ← Updat Option
)
```

1 Objective: To add a new field "location" with value "Newark" to the document (_id:4) of "Students" Collection

Ilp: Check the document (_id:4) in the "Students" Collection before Proceeding.

```
>db.Students.find({_id:4}).pretty();
{
  "_id":4,
  "Grade":"VII",
  "StudName":"Hesch Gibbs",
  "Hobbies":"Graffiti"
}
>
```

Act:

```
>db.Students.Updat({_id:4},{$set:{location:"Newark"}});
WriteResult({"nMatched":1,"nUpserted":0,"nModified":1})
>
```

Outcome: Confirm that the new field "location" with the value "Newark" has been added to document (_id:4) in the "Students" Collection

```
>db.Students.find({_id:4}).pretty();
{ "_id":4,
  "Grade":"VII"
  "StudName":"Hesch Gibbs",
```

```
"Hobbies" : "Graffiti",
"location" : "Newark"
}
>
```

## 6.5.4 Removing an Existing field from an Existing Document – Remove method

Syntax:

```
db.Students.remove(        <-- Collection
{Age : {$gt 18}},          <-- Remove Criteria
)
```

1) Objective:
To remove the field "location" with value "Newark" in the document (_id:4) of "Students" Collection.

I/p

```
> db.Students.find({_id:4}).pretty();
{
    "_id" : 4,
    "Grade" : "VII",
    "StudName" : "Hersch Gibbs",
    "Hobbies" : "Graffiti",
    "location" : "Newark"
}
>
```

Act

```
> db.Students.update({_id:4}, {$unset:{location:"Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1})
>
```

Outcome: Confirm the stated field ("location") has been dropped from the document (_id:4) of the "Students" collection.

### 6.5.4 Removing an Existing Field from an Existing Document – Remove Method

In this section we will explain the syntax of remove method.

```
db.students.remove(      ◄———— Collection
{Age: {$gt 18}},         ◄———— Remove Criteria
)
```

**Objective:** To remove the field "Location" with value "Newark" in the document (_id:4) of "Students" collection.

**Input:** Check the document (_id:4) in the "Students" collection before proceeding.

```
> db.Students.find({_id:4}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti",
        "Location" : "Newark"
}
>
```

**Act:**

```
db.Students.update({_id:4},{$unset:{Location:"Newark"}});
```

```
> db.Students.update({_id:4},{$unset:{Location:"Newark"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**Outcome:** Confirm if the stated field ("Location") has been dropped from the document (_id:4) of the "Students" collection.

```
> db.Students.find({_id:4}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
>
```

### 6.5.5 Finding Documents based on Search Criteria – Find Method

The syntax of find method is as follows:

```
db.students.find(                ◄———— Collection
{Age: {$gt 18}},                 ◄———— Selection Criteria
{RollNo:1,Age:1,_id:1}           ◄———— Projection
).limit(10)                      ◄———— Cursor Modifier
```

**Objective:** To search for documents from the "Students" collection based on certain search criteria.

**Input:** Check the documents in the "Students" collection before proceeding.

```
> db.Students.find({}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"

        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"

        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"

        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"

        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**Act:** Find the document wherein the "StudName" has value "Aryan David".

**db.Students.find({StudName:"Aryan David"});**

**Outcome:**

```
> db.Students.find({StudName:"Aryan David"});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
>
```

To format the above output, use the pretty() method:

**db.Students.find({StudName:"Aryan David"}).pretty();**

```
> db.Students.find({StudName:"Aryan David"}).pretty();
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
```

**RDBMS equivalent:**

```
Select *
    From Students
        Where StudName like 'Aryan David';
```

```
SQL> select * from Students where StudName like 'Aryan David';

STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ------------------------------
3     Aryan David                 VII   Chess

SQL>
```

**Objective:** To display only the StudName from all the documents of the Student's collection. The identifier "__id" should be suppressed and NOT displayed.

**Act:**

**db.Students.find({},{StudName:1,_id:0});**

**Outcome:**

```
> db.Students.find({},{StudName:1,_id:0});
{ "StudName" : "Michelle Jacintha" }
{ "StudName" : "Aryan David" }
{ "StudName" : "Hersch Gibbs" }
{ "StudName" : "Vamsi Bapat" }
{ "StudName" : "Mabel Mathews" }
```

**RDBMS equivalent:**

Select StudName
From Students;

```
SQL> select StudName from Students;

STUDNAME
------------------------
Michelle Jacintha
Aryan David
Mabel Mathews
Hersch Gibbs
Vamsi Bapat

SQL>
```

**Objective:** To display only the StudName and Grade from all the documents of the Students collection. The identifier _id should be suppressed and NOT displayed.

**Act:**

**db.Students.find({},{StudName:1,Grade:1,_id:0});**

**Outcome:**

```
> db.Students.find({},{StudName:1,Grade:1,_id:0});
{ "StudName" : "Michelle Jacintha", "Grade" : "VII" }
{ "Grade" : "VII", "StudName" : "Aryan David" }
{ "Grade" : "VII", "StudName" : "Hersch Gibbs" }
{ "StudName" : "Vamsi Bapat", "Grade" : "VI" }
{ "StudName" : "Mabel Mathews", "Grade" : "VII" }
```

**RDBMS equivalent:**

Select StudName, Grade
From Students;

```
SQL> select StudName, Grade from Students;

STUDNAME                 GRADE
------------------------ -----
Michelle Jacintha        VII
Aryan David              VII
Mabel Mathews            VII
Hersch Gibbs             VII
Vamsi Bapat              VI

SQL>
```

**Objective:** To display the StudName, Grade as well the identifier, _id from the document of the Students collection where the _id column is 1.

**Act:**

**db.Students.find({_id:1},{StudName:1,Grade:1});**

**Outcome:**

```
> db.Students.find({_id:1},{StudName:1,Grade:1});
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII" }
>
```

**RDBMS equivalent:**

Select StudRollNo, StudName, Grade
From Students
Where StudRollNo = '1';

```
SQL> select StudRollNo, StudName, Grade from Students where StudRollNo = '1';
STUDR STUDNAME                    GRADE
----- --------------------------- -----
1     Michelle Jacintha           VII
SQL>
```

**Objective:** To display the StudName and Grade from the document of the Students collection where the _id column is 1. The _id field should NOT be displayed.

**Act:**

```
db.Students.find({_id:1},{StudName:1,Grade:1,_id:0});
```

**Outcome:**

```
> db.Students.find({_id:1},{StudName:1,Grade:1,_id:0});
{ "StudName" : "Michelle Jacintha", "Grade" : "VII" }
>
```

**RDBMS equivalent:**

Select StudName, Grade
From Students
Where StudRollNo like '1';

```
SQL> select StudName, Grade from Students where StudRollNo like '1';
STUDNAME                  GRADE
------------------------- -----
Michelle Jacintha         VII
SQL>
```

***Relational operators available to use in the search criteria:***

| | |
|---|---|
| $eq | → equal to |
| $ne | → not equal to |
| $gte | → greater than or equal to |
| $lte | → less than or equal to |
| $gt | → greater than |
| $lt | → less than |

**Objective:** To find those documents where the Grade is set to 'VII'.

**Act:**

```
db.Students.find({Grade:{$eq:'VII'}}).pretty();
```

**Outcome:**

```
> db.Students.find({Grade:{$eq:'VII'}}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 2,
        "StudName" : "Nabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**RDBMS Equivalent:**

Select *

From Students

Where Grade like 'VII';

```
SQL> select * from Students where Grade like 'VII';
STUDR STUDNAME                 GRADE HOBBIES
----- ------------------------ ----- ------------------------------
    1 Michelle Jacintha        VII   Internet surfing
    3 Aryan David              VII   Chess
    2 Nabel Mathews            VII   Baseball
    4 Hersch Gibbs             VII   Graffiti
SQL>
```

**Objective:** To find those documents where the Grade is NOT set to 'VII'.

**Act:**

**db.Students.find({Grade:{$ne:'VII'}}).pretty();**

**Outcome:**

```
> db.Students.find({Grade:{$ne:'VII'}}).pretty();
{
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"
}
```

There is just one document that meets the above criteria of Grade NOT EQUAL to 'VII'.

**RDBMS Equivalent:**

Select *

From Students

Where Grade <> 'VII';

```
SQL> select * from Students where Grade <> 'VII';
STUDR STUDNAME                 GRADE HOBBIES
----- ------------------------ ----- ------------------------------
    5 Vamsi Bapat              VI
SQL>
```

OR

```
SQL> select * from Students where Grade != 'VII';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ------------------------------
      Vamsi Bapat                 VI
SQL>
```

**Objective:** To find those documents from the Students collection where the Hobbies is set to either 'Chess' or is set to 'Skating'.

**Act:**

```
db.Students.find ({Hobbies :{ $in: ['Chess','Skating']}}).pretty ();
```

**Outcome:**

```
> db.Students.find ({Hobbies :{ $in: ['Chess','Skating']}}).pretty ();
{
        "_id" : 3,
        "Grade" : "VII",
        "Studname" : "Aryan David",
        "Hobbies" : "Chess"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where Hobbies in ('Chess', 'Skating');
```

```
SQL> select * from Students where Hobbies in ('Chess', 'Skating');
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ------------------------------
3     Aryan David                 VII   Chess
SQL>
```

**Objective:** To find those documents from the Students collection where the Hobbies is set neither to 'Chess' nor is set to 'Skating'.

**Act:**

```
db.Students.find ({Hobbies :{ $nin: ['Chess','Skating']}}).pretty ();
```

**Outcome:**

```
> db.Students.find ({Hobbies :{ $nin: ['Chess','Skating']}}).pretty ();
{
        "_id" : 1,
        "Studname" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "Studname" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : ObjectId("54648498889adlab07d4896ff"),
        "Studname" : "Vamsi Bapat",
        "Grade" : "VI"
}
{
        "_id" : 2,
        "Studname" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where Hobbies not in ('Chess', 'Skating');
```

```
SQL> select * from Students where Hobbies not in ('Chess','Skating');

STUDR STUDNAME                 GRADE HOBBIES
----- ------------------------ ----- ------------------------------
1     Michelle Jacintha        VII   Internet surfing
2     Mabel Mathews            VII   Baseball
4     Hersch Gibbs             VII   Graffiti

SQL>
```

**Objective:** To find those documents from the Students collection where the Hobbies is set to 'Graffiti' and the StudName is set to 'Hersch Gibbs' (AND condition).

**Act:**

```
db.Students.find({Hobbies:'Graffiti', StudName: 'Hersch Gibbs'}).pretty();
```

**Outcome:**

```
> db.Students.find({Hobbies:'Graffiti', StudName: 'Hersch Gibbs'}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
>
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where Hobbies like 'Graffiti' and StudName like 'Hersch Gibbs';
```

```
SQL> select * from Students where Hobbies like 'Graffiti' and StudName like 'Her
sch Gibbs';

STUDR STUDNAME                 GRADE HOBBIES
----- ------------------------ ----- ------------------------------
4     Hersch Gibbs             VII   Graffiti

SQL>
```

**Objective:** To find documents from the Students collection where the StudName begins with "M".

**Act:**

```
db.Students.find({StudName:/^M/}).pretty();
```

**Outcome:**

```
> db.Students.find({StudName:/^M/}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
>
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where StudName like 'M%';
```

```
SQL> select * from Students where StudName like 'M%';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ---------------------------------------
1     Michelle Jacintha           VII   Internet surfing
2     Mabel Mathews               VII   Baseball
SQL>
```

**Objective:** To find documents from the Students collection where the StudName ends in "s".

**Act:**

```
db.Students.find({StudName:/s$/}).pretty();
```

**Outcome:**

```
> db.Students.find({StudName:/s$/}).pretty();
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where StudName like '%s';
```

```
SQL> select * from Students where StudName like '%s';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ---------------------------------------
2     Mabel Mathews               VII   Baseball
4     Hersch Gibbs                VII   Graffiti
SQL>
```

**Objective:** To find documents from the Students collection where the StudName has an "e" in any position.

**Act:**

```
db.Students.find({StudName:/e/}).pretty();

                                        OR

db.Students.find({StudName:/.*e.*/}).pretty();

                                        OR

db.Students.find({StudName:{$regex:"e"}}).pretty();
```



**Outcome:**

```
> db.Students.find({StudName:/e/}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where StudName like '%e%';
```

```
SQL> select * from Students where StudName like '%e%';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ---------------------------------------
1     Michelle Jacintha           VII   Internet surfing
2     Mabel Mathews               VII   Baseball
4     Hersch Gibbs                VII   Graffiti
SQL>
```

**Objective:** To find documents from the Students collection where the StudName ends in "a".

**Act:**

```
db.Students.find({StudName:{$regex:"a$"}}).pretty();
```

**Outcome:**

```
> db.Students.find({StudName:{$regex:"a$"}}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where StudName like '%a';
```

```
SQL> select * from Students where StudName like '%a';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ---------------------------------------
1     Michelle Jacintha           VII   Internet surfing
SQL>
```

**Objective:** To find documents from the Students collection where the StudName begins with "M"

**Act:**

```
db.Students.find({StudName:{$regex:"^M"}}).pretty();
```

**Outcome:**

```
> db.Students.find({StudName:{$regex:"^M"}}).pretty();
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Where StudName like 'M%';
```

```
SQL> select * from Students where StudName like 'M%';
STUDR STUDNAME                    GRADE HOBBIES
----- --------------------------- ----- ------------------------------
1     Michelle Jacintha           VII   Internet surfing
2     Mabel Mathews               VII   Baseball
SQL>
```

## 6.5.6 Dealing with NULL Values

**Objective:** To add a new field with null value in existing documents (_id:3 and _id:4) of Students collection. A NULL is a missing or unknown value. When we place NULL as a value for a field, it implies that currently we do not know the value or the value is missing. We can always update the value of the field once we know it.

**Input:** Before we execute the commands to update documents with a null value in a column, let us first view the two documents.

```
db.Students.find({$or:[{_id:3},{_id:4}]})
```

```
> db.Students.find({$or:[{_id:3},{_id:4}]});
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess" }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti" }
```

**Act:** Update the documents with NULL values in the "Location" column.

```
db.Students.update({_id:3},{$set:{Location:null}});
db.Students.update({_id:4},{$set:{Location:null}});
```

```
> db.Students.update({_id:3},{$set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.Students.update({_id:4},{$set:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**RDBMS Equivalent:**

```
Update Students
        Set Location = null
                Where StudRollNo in ('3','4');
```

```
SQL> update Students set Location = null where StudRollNo in ('3','4');
2 rows updated.
SQL>
```

**Outcome:** To search for NULL values in Location column.

**db.Students.find({Location:{$eq:null}});**

```
> db.Students.find({Location:{$eq:null}});
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess", "Location" : null }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti", "Location" : null }
{ "_id" : ObjectId("5464849889ad1ab07d489b7f"), "StudName" : "Vansi Bapat", "Grade" : "VI" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
```

The above statement displays documents which either have NULL values in the location column or do not have the location column at all.

**RDBMS Equivalent:**

```
Select *
        From Students
                Where Location is Null;
```

```
SQL> select * from Students where Location is NULL;

STUDR STUDNAME                    GRADE HOBBIES                        LOCATION
----- --------------------------- ----- ------------------------------ ----------
    1 Michelle Jacintha           VII   Internet surfing
    3 Aryan David                 VII   Chess
    2 Mabel Mathews               VII   Baseball
    4 Hersch Gibbs                VII   Graffiti
      Vansi Bapat                 VI

SQL>
```

**Objective:** To remove "Location" field having "NULL" values from the documents (_id:3 and _id:4) from the Students collection.

**Input:** Document from the "Students" collection having "NULL" values in the "Location" column.

```
> db.Students.find({Location:{$eq:null}});
{ "_id" : 1, "StudName" : "Michelle Jacintha", "Grade" : "VII", "Hobbies" : "Internet Surfing" }
{ "_id" : 3, "Grade" : "VII", "StudName" : "Aryan David", "Hobbies" : "Chess", "Location" : null }
{ "_id" : 4, "Grade" : "VII", "StudName" : "Hersch Gibbs", "Hobbies" : "Graffiti", "Location" : null }
{ "_id" : ObjectId("5464849889ad1ab07d489b7f"), "StudName" : "Vansi Bapat", "Grade" : "VI" }
{ "_id" : 2, "StudName" : "Mabel Mathews", "Grade" : "VII", "Hobbies" : "Baseball" }
```

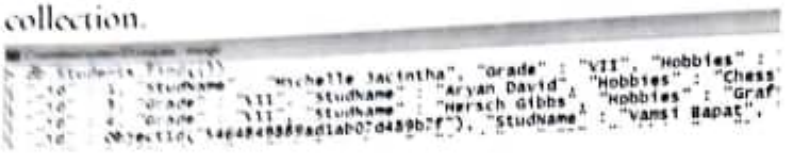
**Act:**

**db.Students.update({_id:3},{$unset:{Location:null}});**

**db.Students.update({_id:4},{$unset:{Location:null}});**

```
> db.Students.update({_id:3},{$unset:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 0 })
> db.Students.update({_id:4},{$unset:{Location:null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 0 })
>
```

**Outcome:** Let us confirm if the changes have been made by running find method on the Students collection.



## 6.5.7 Count, Limit, Sort, and Skip

**Objective:** To find the number of documents in the Students collection.

**Act:**

```
db.Students.count()
```

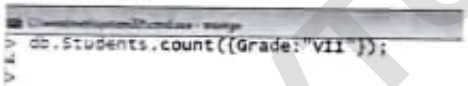
**Outcome:**


**Objective:** To find the number of documents in the Students collection wherein the Grade is VII.

**Act:**

```
db.Students.count({Grade:"VII"});
```

**Outcome:**

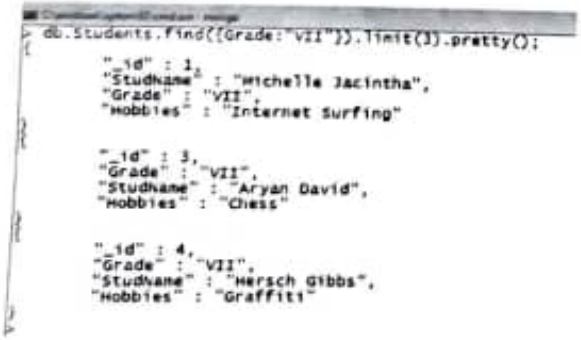



**Objective:** To retrieve the first 3 documents from the Students collection wherein the Grade is VII.

**Act:**

```
db.Students.find({Grade:"VII"}).limit(3).pretty();
```

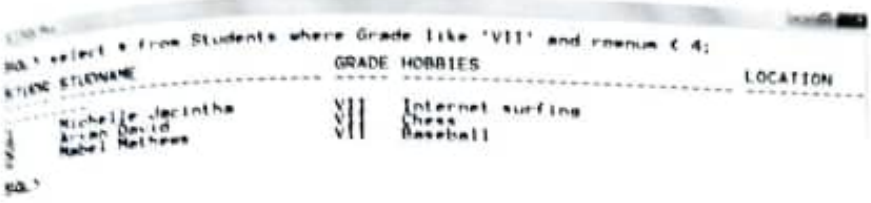
**Outcome:**




***RDBMS Equivalent:***

Select *

From Students
Where Grade like 'VII' and rownum < 4;



**Objective:** To sort the documents from the Students collection in the ascending order of StudName.

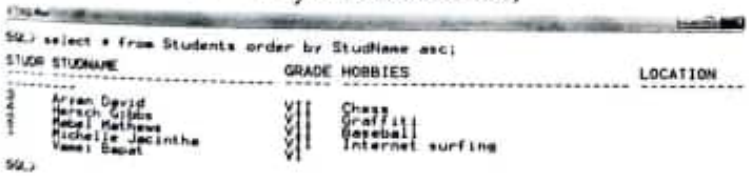
**Act:**

```
db.Students.find().sort({StudName:1}).pretty();
```

**Outcome:**

**RDBMS Equivalent:**

Select *
From Students
Order by StudName asc;



**Objective:** To sort the documents from the Students collection in the descending order of StudName.

**Act:**

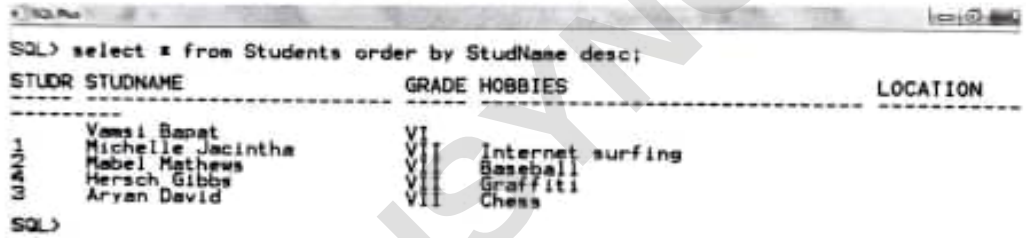
```
db.Students.find().sort({StudName:-1}).pretty();
```

**Outcome:**

```
db.Students.find().sort({StudName:-1}).pretty();
{
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"
}
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Order by StudName desc;
```



```
SQL> select * from Students order by StudName desc;

STUDR STUDNAME                    GRADE HOBBIES                         LOCATION
----- --------------------------- ----- ------------------------------- ----------
---------
      Vamsi Bapat                 VI
      Michelle Jacintha           VII   Internet surfing
      Mabel Mathews               VII   Baseball
      Hersch Gibbs                VII   Graffiti
      Aryan David                 VII   Chess

SQL>
```

**Objective:** To sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in descending order.

**Act:**

**db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();**

**Outcome:**

```
db.Students.find().sort({Grade:1, Hobbies:-1}).pretty();
{
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"
}
{
        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
}
{
        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"
}
{
        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"
}
{
        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"
}
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Order by Grade asc, hobbies desc;
```

```
SQL> select * from Students order by Grade asc, Hobbies desc;
STUDR STUDNAME                    GRADE HOBBIES                          LOCATION
----- --------------------------- ----- -------------------------------- ----------
      Vamsi Bapat                 VI
      Michelle Jacintha           VII   Internet surfing
      Hersch Gibbs                VII   Graffiti
      Aryan David                 VII   Chess
      Mabel Mathews               VII   Baseball

SQL>
```

**Objective:** To sort the documents from the Students collection first on Grade in ascending order and then on Hobbies in ascending order.

**Act:**

```
db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
```

**Outcome:**

```
db.Students.find().sort({Grade:1, Hobbies:1}).pretty();
        "_id" : ObjectId("5464849889ad1ab07d489b7f"),
        "StudName" : "Vamsi Bapat",
        "Grade" : "VI"

        "_id" : 2,
        "StudName" : "Mabel Mathews",
        "Grade" : "VII",
        "Hobbies" : "Baseball"

        "_id" : 3,
        "Grade" : "VII",
        "StudName" : "Aryan David",
        "Hobbies" : "Chess"

        "_id" : 4,
        "Grade" : "VII",
        "StudName" : "Hersch Gibbs",
        "Hobbies" : "Graffiti"

        "_id" : 1,
        "StudName" : "Michelle Jacintha",
        "Grade" : "VII",
        "Hobbies" : "Internet Surfing"
```

**RDBMS Equivalent:**

```
Select *
        From Students
                Order by Grade asc, Hobbies asc;
```

```
SQL> select * from Students order by Grade asc, Hobbies asc;
STUDR STUDNAME                    GRADE HOBBIES                          LOCATION
----- --------------------------- ----- -------------------------------- ----------
      Vamsi Bapat                 VI
      Mabel Mathews               VII   Baseball
      Aryan David                 VII   Chess
      Hersch Gibbs                VII   Graffiti
      Michelle Jacintha           VII   Internet surfing
SQL>
```

### 6.5.8 Arrays

**Objective:** To create a collection by the name "food" and then insert documents into the "food" collection. Each document should have a "fruits" array.

**Act:**

```
db.food.insert({_id:1,fruits:[ 'banana','apple','cherry' ] })
db.food.insert({_id:2,fruits:[ 'orange','butterfruit','mango' ]})
db.food.insert({_id:3,fruits:[ 'pineapple','strawberry','grapes']});
db.food.insert({_id:4,fruits:[ 'banana','strawberry','grapes']});
db.food.insert({_id:5,fruits:[ 'orange','grapes']});
```

```
> db.food.insert({_id:1,fruits:[ 'banana','apple','cherry' ] })
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:2,fruits:[ 'orange','butterfruit','mango' ]})
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:3,fruits:[ 'pineapple','strawberry','grapes']});
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:4,fruits:[ 'banana','strawberry','grapes']});
WriteResult({ "nInserted" : 1 })
> db.food.insert({_id:5,fruits:[ 'orange','grapes']});
WriteResult({ "nInserted" : 1 })
>
```

**Outcome:** Let us check if these documents are now in the "food" collection.

```
db.food.find({})
```

```
> db.food.find({})
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
{ "_id" : 2, "fruits" : [ "orange", "butterfruit", "mango" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "strawberry", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
>
```

**Objective:** To find those documents from the "food" collection which has the "fruits array" constituted of "banana", "apple" and "cherry".

**Act:**

```
db.food.find({fruits:['banana','apple','cherry']}).pretty()
```

**Outcome:**

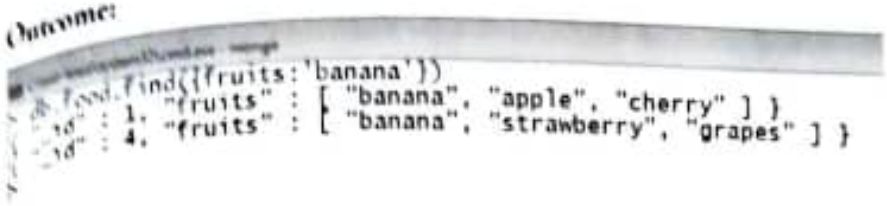
```
> db.food.find({fruits:['banana','apple','cherry']}).pretty()
{ "_id" : 1, "fruits" : [ "banana", "apple", "cherry" ] }
>
```

**Objective:** To find those documents from the "food" collection which has the "fruits" array having "banana", as an element.

**Act:**

```
db.food.find({fruits:'banana'})
```

**Outcome:**




**Objective:** To find those documents from the "food" collection which have the "fruits" array having "grapes" in the first index position. The index position begins at 0.

**Act:**

```
db.food.find({'fruits.1':'grapes'})
```

**Outcome:**


**Objective:** To find those documents from the "food" collection where "grapes" is present in the 2nd index position of the "fruits" array.

**Act:**

```
db.food.find({'fruits.2':'grapes'})
```

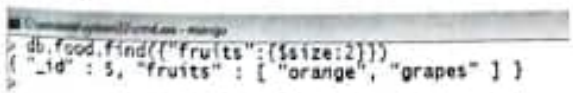
**Outcome:**


**Objective:** To find those documents from the "food" collection where the size of the array is two. The size implies that the array holds only 2 values.

**Act:**

```
db.food.find({"fruits":{$size:2}})
```

**Outcome:**




**Objective:** To find those documents from the "food" collection where the size of the array is three. The size implies that the array holds only 3 values.

**Act:**

```
db.food.find({"fruits":{$size:3}})
```

**Outcome:**

**Objective:** To find the document with (_id: 1) from the "food" collection and display the first two elements from the array "fruits".

**Act:**

```
db.food.find({_id:1},{"fruits":{$slice:2}})
```

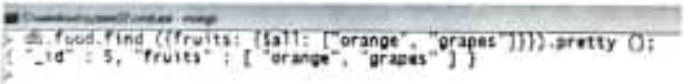
**Outcome:**

**Objective:** To find all documents from the "food" collection which have elements "orange" and "grapes" in the array "fruits".

**Act:**

```
db.food.find ({fruits: {$all: ["orange", "grapes"]}}).pretty ();
```

**Outcome:**

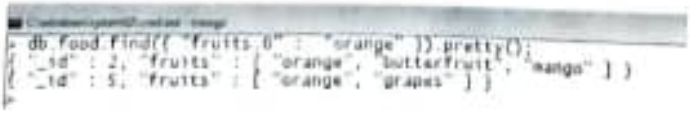


**Objective:** To find those documents from the "food" collection which have the element "orange" in the 0th index position in the array "fruits".

**Act:**

```
db.food.find({ "fruits.0" : "orange" }).pretty();
```

**Outcome:**



### 6.5.8.2 Further Updates to the Array "fruits" ...

Before we do the updates to the documents in the food collection, let us look at the current state:

```
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
        "_id" : 2,
        "fruits" : [
                "orange",
                "butterfruit",
                "mango"
        ],
        "price" : [
                {
                        "orange" : 60,
                        "butterfruit" : 200,
                        "mango" : 120
                }
        ]
}
>
```

**Objective:** To update the document with "_id:4" by adding an element "orange" to the list of elements in the array "fruits".

**Act:**

```
db.food.update({_id:4},{$addToSet:{fruits:"orange"}});
```

```
> db.food.update({_id:4},{$addToSet:{fruits:"orange"}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**Outcome:** The result after the execution of the statement is as follows.

```
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes", "orange" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
        "_id" : 2,
        "fruits" : [
                "orange",
                "butterfruit",
                "mango"
        ],
        "price" : [
                {
                        "orange" : 60,
                        "butterfruit" : 200,
                        "mango" : 120
                }
        ]
}
>
```

**Objective:** To update the document with "_id:4" by popping an element from the list of elements present in the array "fruits". The element popped is the one from the end of the array.

**Act:**

```
db.food.update({_id:4},{$pop:{fruits:1}});
```

```
> db.food.update({_id:4},{$pop:{fruits:1}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**Outcome:** The "food" collection after the execution of the statement is as follows.

```
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "banana", "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
        "_id" : 2,
        "fruits" : [
                "orange",
                "butterfruit",
                "mango"
        ],
        "price" : [
                {
                        "orange" : 60,
                        "butterfruit" : 200,
                        "mango" : 120
                }
        ]
}
```

**Objective:** To update the document with "_id:4" by popping an element from the list of elements present in the array "fruits". The element popped is the one from the beginning of the array.

**Act:**

```
db.food.update({_id:4},{$pop:{fruits:-1}});
```

```
> db.food.update({_id:4},{$pop:{fruits:-1}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**Outcome:** The "food" collection after the execution of the above update statement is as follows.

```
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
        "_id" : 2,
        "fruits" : [
                "orange",
                "butterfruit",
                "mango"
        ],
        "price" : [
                {
                        "orange" : 60,
                        "butterfruit" : 200,
                        "mango" : 120
                }
        ]
}
```

**Objective:** To update the document with "_id:3" by popping two elements from the list of elements present in the array "fruits". The elements popped are "pineapple" and "grapes".

The document with "_id:3" before the update is

```
> db.food.find({_id:3});
{ "_id" : 3, "fruits" : [ "pineapple", "strawberry", "grapes" ] }
>
```

**Act:**

```
db.food.update({_id:3},{$pullAll:{fruits: [ 'pineapple','grapes' ]}});
```

```
{ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
```

**Outcome:** The document with "_id:3" after the update is as follows:

```
> db.food.find({_id:4});
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }
```

**Objective:** To update the documents having "banana" as an element in the array "fruits" and pop out the element "banana" from those documents.

The "food" collection before the update is as follows:

```
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "banana", "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "strawberry" ] }
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
        "_id" : 2,
        "fruits" : [
                "orange",
                "butterfruit",
                "mango"
        ],
        "price" : [
                {
                        "orange" : 60,
                        "butterfruit" : 200,
                        "mango" : 120
                }
        ]
}
>
```

**Act:**

```
db.food.update({fruits:'banana'}, {$pull:{fruits:'banana'}})
```

```
> db.food.update({fruits:'banana'}, {$pull:{fruits:'banana'}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**Outcome:** The "food" collection after the update is as follows:

```
> db.food.find().pretty();
{ "_id" : 1, "fruits" : [ "An apple", "cherry" ] }
{ "_id" : 3, "fruits" : [ "strawberry" ] }
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }
{ "_id" : 5, "fruits" : [ "orange", "grapes" ] }
{
        "_id" : 2,
        "fruits" : [
                "orange",
                "butterfruit",
                "mango"
        ],
        "price" : [
                {
                        "orange" : 60,
                        "butterfruit" : 200,
                        "mango" : 120
                }
        ]
}
>
```

**Objective:** To pull out an array element based on index position.

There is no direct way of pulling the array elements by looking up their index numbers. However a workaround is available. The document with "_id:4" in the food collection prior to the update is as follows:

```
> db.food.find({_id:4}).pretty();
{ "_id" : 4, "fruits" : [ "apple", "grapes" ] }
>
```

**Act:** The update statement is

```
db.food.update({_id:4}, {$unset : {"fruits.1" : null }}) ;
db.food.update({_id:4}, {$pull : {"fruits" : null}});
```

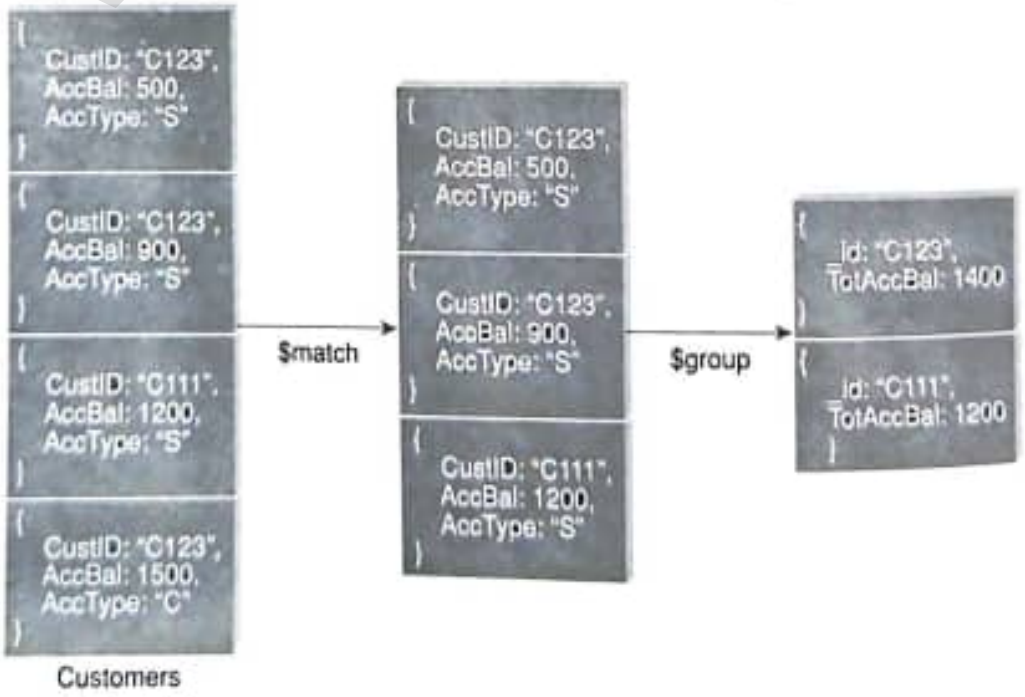
```
> db.food.update({_id:4}, {$unset : {"fruits.1" : null }}) ;
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
> db.food.update({_id:4}, {$pull : {"fruits" : null}});
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
```

**Outcome:** After update, the document with _id:4 in the food collection is

```
> db.food.find({_id:4}).pretty();
{ "_id" : 4, "fruits" : [ "apple" ] }
>
```

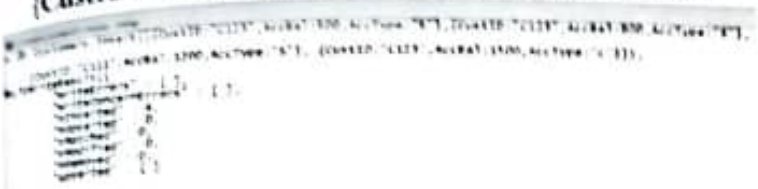
### 6.5.9 Aggregate Function

**Objective:** Consider the collection "Customers" as given below. It has four documents. We would like to filter out those documents where the "AccType" has a value other than "S". After the filter, we should be left with three documents where the "Acctype": "S". It is then required to group the documents on the basis of CustID and sum up the "AccBal" for each unique "CustID". This is similar to the output received with group by clause in RDBMS. Once the groups have been formed [as per the example below, there will be only two groups: (a) "CustID" : "C123" and (b) "CustID" : "C111"], filter and display that group where the "TotAccBal" column has a value greater than 1200.



Let us start off by creating the collection "Customers" with the above displayed four documents:

```
db.Customers.insert([{CustID:"C123",AccBal:500,AccType:"S"},
{CustID:"C123", AccBal: 900, AccType:"S"},
{CustID:"C111", AccBal: 1200, AccType:"S"},
{CustID:"C123", AccBal: 1500, AccType:"C"}]);
```

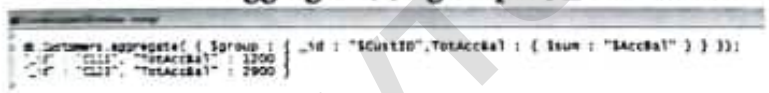


To confirm the presence of four documents in the "Customers" collection, use the below syntax:

```
db.Customers.find().pretty();
```

To group on "CustID" and compute the sum of "AccBal", use the below syntax:

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } });
```

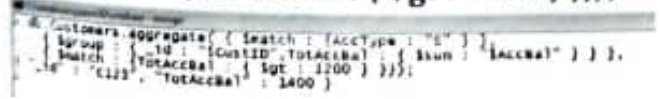


In order to first filter on "AccType:S" and then group it on "CustID" and then compute the sum of "AccBal", use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } });
```

In order to first filter on "AccType:S" and then group it on "CustID" and then to compute the sum of "AccBal" and then filter those documents wherein the "TotAccBal" is greater than 1200, use the below syntax:

```
db.Customers.aggregate( { $match : {AccType : "S" } },
{ $group : { _id : "$CustID",TotAccBal : { $sum : "$AccBal" } } },
{ $match : {TotAccBal : { $gt : 1200 } }});
```



**To group on "CustID" and compute the average of the "AccBal" for each group:**

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $avg : "$AccBal" } } });
```

**To group on "CustID" and determine the maximum "AccBal" for each group:**

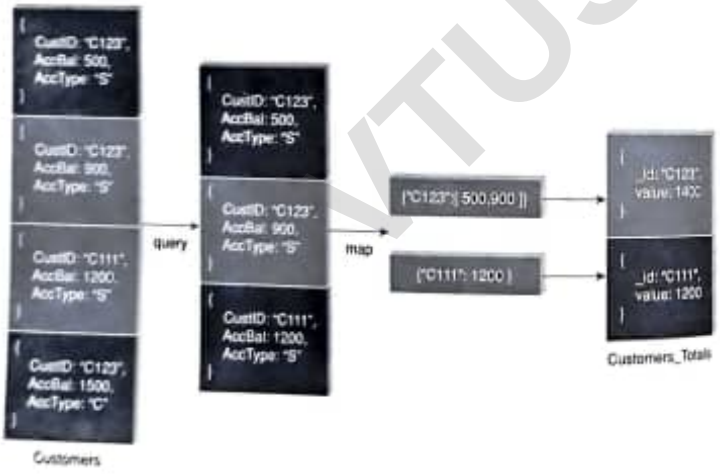
```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $max : "$AccBal" } } });
```

**To group on "CustID" and determine the minimum "AccBal" for each group:**

```
db.Customers.aggregate( { $group : { _id : "$CustID",TotAccBal : { $min : "$AccBal" } } });
```

### 6.5.10 MapReduce Function

**Objective:** Consider the collection "Customers" below. There are four documents. Run a query to filter out those documents where the key "AccType" has a value other than "S". Then for each unique CustID, prepare a list of AccBal values. For example, for CustID: "C123", the AccBals are 500,900. This task will be assigned to the mapper function. The output from the mapper function serves as the input to the reducer function. The reducer function then aggregates the AccBal for each CustID. For example, for CustID: "C123", the value is 1400, etc.






Given below is the syntax that we will use to accomplish the objective.

```
db.Customers.mapReduce (
map       →   function() { emit ( this.CustID, this.AccBal ); },
reduce    →   function(key, values) { return Array.sum (values ) },
          {
query     →   query: { AccType: "S"},
output    →   out: "Customer_Totals"
          }
          )
```

***Map Function***

```
var map = function(){
emit ( this.CustID, this.AccBal );}
```

***Reduce Function***

```
var reduce = function(key, values){ return Array.sum(values) ; }
```

***To execute the query***

```
db.Customers.mapReduce(map, reduce,{out: "Customer_Totals", query:{AccType:"S"}});
```

```
{
        "result" : "Customer_Totals",
        "timeMillis" : 7,
        "counts" : {
                "input" : 3,
                "emit" : 3,
                "reduce" : 1,
                "output" : 2
        },
        "ok" : 1,
}
```

***The output as archived in "Customer_Totals" collection:***

```
{ "_id" : "C111", "value" : 1200 }
{ "_id" : "C123", "value" : 1400 }
```

### 6.5.11 Java Script Programming

**Objective:** To compute the factorial of a given positive number. The user is required to create a function by the name "factorial" and insert it into the "system.js" collection.

Before we proceed, a quick check on what is contained in the "system.js" collection:

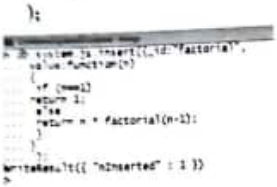
```
db.system.js.find()
```

As per the screenshot above, currently there are no functions in the system.js collection.

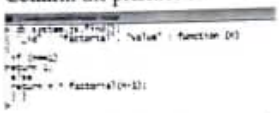
**Act:**

```
db.system.js.insert({_id:"factorial",
```

```
value:function(n)
{
    if (n==1)
    return 1;
    else
    return n * factorial(n-1);
}
    }
);
```



Confirm the presence of the "factorial" function in the system.js collection.



To execute the function "factorial", use the eval() method.

```
db.eval("factorial(3)");
```

```
db.eval("factorial(5)");
```

```
db.eval("factorial(1)");
```

### 6.5.12 Cursors in MongoDB

**Objective:** To create a collection by the name "alphabets" and insert documents in it containing two fields, "_id" and "alphabet". The values stored in the "alphabet" field should be "a", "b", "c", "d", etc. with one value stored per document. There should be 26 documents in all. We need to use cursor to iterate through the "alphabets" collection.

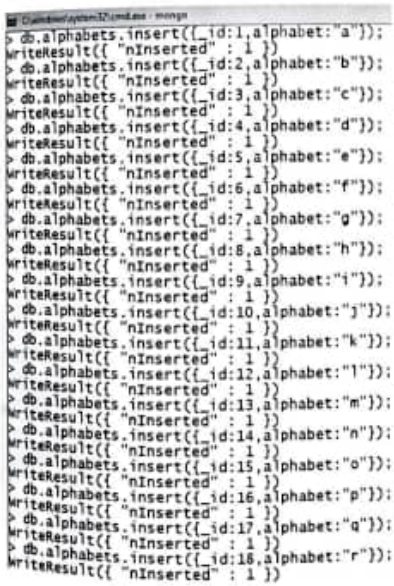
***Note:*** "Alphabets" is the name of the collection and "alphabet" is the name of the field.

**Act:** To create the collection "alphabets" with its 26 documents.

```
db.alphabets.insert({_id:1,alphabet:"a"});
db.alphabets.insert({_id:2,alphabet:"b"});
```



```
db.alphabets.insert({_id:3,alphabet:"c"});
db.alphabets.insert({_id:4,alphabet:"d"});
db.alphabets.insert({_id:5,alphabet:"e"});
db.alphabets.insert({_id:6,alphabet:"f"});
db.alphabets.insert({_id:7,alphabet:"g"});
db.alphabets.insert({_id:8,alphabet:"h"});
db.alphabets.insert({_id:9,alphabet:"i"});
db.alphabets.insert({_id:10,alphabet:"j"});
db.alphabets.insert({_id:11,alphabet:"k"});
db.alphabets.insert({_id:12,alphabet:"l"});
db.alphabets.insert({_id:13,alphabet:"m"});
db.alphabets.insert({_id:14,alphabet:"n"});
db.alphabets.insert({_id:15,alphabet:"o"});
db.alphabets.insert({_id:16,alphabet:"p"});
db.alphabets.insert({_id:17,alphabet:"q"});
db.alphabets.insert({_id:18,alphabet:"r"});
db.alphabets.insert({_id:19,alphabet:"s"});
db.alphabets.insert({_id:20,alphabet:"t"});
db.alphabets.insert({_id:21,alphabet:"u"});
db.alphabets.insert({_id:22,alphabet:"v"});
db.alphabets.insert({_id:23,alphabet:"w"});
db.alphabets.insert({_id:24,alphabet:"x"});
db.alphabets.insert({_id:25,alphabet:"y"});
db.alphabets.insert({_id:26,alphabet:"z"});
```



```
> db.alphabets.insert({_id:19,alphabet:"s"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:20,alphabet:"t"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:21,alphabet:"u"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:22,alphabet:"v"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:23,alphabet:"w"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:24,alphabet:"x"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:25,alphabet:"y"});
WriteResult({ "nInserted" : 1 })
> db.alphabets.insert({_id:26,alphabet:"z"});
WriteResult({ "nInserted" : 1 })
>
```

Confirm the presence of 26 documents in the "alphabets" collection.

```
> db.alphabets.find();
{ "_id" : 1, "alphabet" : "a" }
{ "_id" : 2, "alphabet" : "b" }
{ "_id" : 3, "alphabet" : "c" }
{ "_id" : 4, "alphabet" : "d" }
{ "_id" : 5, "alphabet" : "e" }
{ "_id" : 6, "alphabet" : "f" }
{ "_id" : 7, "alphabet" : "g" }
{ "_id" : 8, "alphabet" : "h" }
{ "_id" : 9, "alphabet" : "i" }
{ "_id" : 10, "alphabet" : "j" }
{ "_id" : 11, "alphabet" : "k" }
{ "_id" : 12, "alphabet" : "l" }
{ "_id" : 13, "alphabet" : "m" }
{ "_id" : 14, "alphabet" : "n" }
{ "_id" : 15, "alphabet" : "o" }
{ "_id" : 16, "alphabet" : "p" }
{ "_id" : 17, "alphabet" : "q" }
{ "_id" : 18, "alphabet" : "r" }
{ "_id" : 19, "alphabet" : "s" }
{ "_id" : 20, "alphabet" : "t" }
type "it" for more
>
```

*A quick word on how the db.collection.find() method works.* This is the primary method for read operation. In other words, it allows one to fetch the documents from the collection. To be able to access the documents, one needs to iterate the cursor.

However, in the mongo shell, if the returned cursor is not assigned to a variable using the var keyword, then cursor is automatically iterated up to 20 times to print the first 20 documents in the result.

```
> var myCursor = db.alphabets.find();
> myCursor;
{ "_id" : 1, "alphabet" : "a" }
{ "_id" : 2, "alphabet" : "b" }
{ "_id" : 3, "alphabet" : "c" }
{ "_id" : 4, "alphabet" : "d" }
{ "_id" : 5, "alphabet" : "e" }
{ "_id" : 6, "alphabet" : "f" }
{ "_id" : 7, "alphabet" : "g" }
{ "_id" : 8, "alphabet" : "h" }
{ "_id" : 9, "alphabet" : "i" }
{ "_id" : 10, "alphabet" : "j" }
{ "_id" : 11, "alphabet" : "k" }
{ "_id" : 12, "alphabet" : "l" }
{ "_id" : 13, "alphabet" : "m" }
{ "_id" : 14, "alphabet" : "n" }
{ "_id" : 15, "alphabet" : "o" }
{ "_id" : 16, "alphabet" : "p" }
{ "_id" : 17, "alphabet" : "q" }
{ "_id" : 18, "alphabet" : "r" }
{ "_id" : 19, "alphabet" : "s" }
{ "_id" : 20, "alphabet" : "t" }
type "it" for more
>
```



Let us now look at designing manual cursors to iterate through the documents in the "alphabets" collection. We will use two methods with manual cursors: hasNext() and next(). We now quickly explain the two methods.

**Method 1:** hasNext() method. Return value: Boolean. The hasNext() method returns true if the cursor returned by the db.Collection.find() query can iterate further to return more documents.

**Method 2:** next() method. The next() method returns the next document in the cursor as returned by the db.collection.find() method.

```
> var myCursor = db.alphabets.find();
> while(myCur.hasNext()){
... var myDocument = myCur.next();
... print("the alphabet is : " + myDoc.alphabet);
... }
the alphabet is : a
the alphabet is : b
the alphabet is : c
the alphabet is : d
the alphabet is : e
the alphabet is : f
the alphabet is : g
the alphabet is : h
the alphabet is : i
the alphabet is : j
the alphabet is : k
the alphabet is : l
the alphabet is : m
the alphabet is : n
the alphabet is : o
the alphabet is : p
the alphabet is : q
the alphabet is : r
the alphabet is : s
the alphabet is : t
the alphabet is : u
the alphabet is : v
the alphabet is : w
the alphabet is : x
the alphabet is : y
the alphabet is : z
>
```

The same result can be obtained by iterating through the cursor using a forEach loop.

```
> var myCursor = db.alphabets.find();
> var myDoc;
> cur.forEach( function(myDoc) {
... print("the alphabet is : " + myDoc.alphabet);
... });
the alphabet is : a
the alphabet is : b
the alphabet is : c
the alphabet is : d
the alphabet is : e
the alphabet is : f
the alphabet is : g
the alphabet is : h
the alphabet is : i
the alphabet is : j
the alphabet is : k
the alphabet is : l
the alphabet is : m
the alphabet is : n
the alphabet is : o
the alphabet is : p
the alphabet is : q
the alphabet is : r
the alphabet is : s
the alphabet is : t
the alphabet is : u
the alphabet is : v
the alphabet is : w
the alphabet is : x
the alphabet is : y
the alphabet is : z
>
```

### 6.5.13 Indexes

Assume the collection with the following documents:

```
> db.books.find().pretty();
{
        "_id" : 6,
        "Category" : "Machine Learning",
        "Bookname" : "Machine Learning for Hackers",
        "Author" : "Drew Conway",
        "qty" : 25,
        "price" : 400,
        "rol" : 30,
        "pages" : 350
}
{
        "_id" : 7,
        "Category" : "Web Mining",
        "Bookname" : "Mining the Social Web",
        "Author" : "Matthew A.Russell",
        "qty" : 55,
        "price" : 500,
        "rol" : 30,
        "pages" : 250
}
{
        "_id" : 8,
        "Category" : "Python",
        "Bookname" : "Python for Data Analysis",
        "Author" : "Wes McKinney",
        "qty" : 8,
        "price" : 150,
        "rol" : 20,
        "pages" : 150
}
{
        "_id" : 9,
        "Category" : "Visualization",
        "Bookname" : "Visualizing Data",
        "Author" : "Ben Fry",
        "qty" : 12,
        "price" : 325,
        "rol" : 6,
        "pages" : 450
}
{
        "_id" : 10,
        "Category" : "Web Mining",
        "Bookname" : "Algorithms for the intelligent web",
        "Author" : "Haralambos Marmanis",
        "qty" : 5,
        "price" : 850,
        "rol" : 10,
        "pages" : 120
}
```

Create an index on the key "Category" in the "books" collection.

```
> db.books.ensureIndex({"Category":1});
{
        "createdCollectionAutomatically" : false,
        "numIndexesBefore" : 1,
        "numIndexesAfter" : 2,
        "ok" : 1
}
```

Check on the status, that is, number and name of the indexes:

```
> db.books.stats();
{
        "ns" : "test.books",
        "count" : 5,
        "size" : 1200,
        "avgObjSize" : 240,
        "storageSize" : 8192,
        "numExtents" : 1,
        "nindexes" : 2,
        "lastExtentSize" : 8192,
        "paddingFactor" : 1,
        "systemFlags" : 1,
```

```
        "userFlags" : 1,
        "totalIndexSize" : 16352,
        "indexSizes" : {
                "_id_" : 8176,
                "Category_1" : 8176
        },
        "ok" : 1
}
```

Get the list of all indexes on the "books" collection:

```
> db.books.getIndexes();
[
        {
                "v" : 1,
                "key" : {
                        "_id" : 1
                },
                "name" : "_id_",
                "ns" : "test.books"
        },
        {
                "v" : 1,
                "key" : {
                        "Category" : 1
                },
                "name" : "Category_1",
                "ns" : "test.books"
        }
]
>
```

To use the index on "Category" in the "books" collection, use the hint method:

```
> db.books.find({"Category":"Web Mining"}).pretty().hint({"Category":1});
{
        "_id" : 7,
        "Category" : "Web Mining",
        "Bookname" : "Mining the Social Web",
        "Author" : "Matthew A.Russell",
        "qty" : 55,
        "price" : 500,
        "rol" : 30,
        "pages" : 250
}
{
        "_id" : 10,
        "Category" : "Web Mining",
        "Bookname" : "Algorithms for the intelligent web",
        "Author" : "Haralambos Marmanis",
        "qty" : 5,
        "price" : 850,
        "rol" : 10,
        "pages" : 120
}
>
```

Check the explain plan to get a deeper understanding on the use of index.

```
> db.books.find({"Category":"Web Mining"}).pretty().hint({"Category":1}).explain();
{
        "cursor" : "BtreeCursor Category_1",
        "isMultiKey" : false,
        "n" : 2,
        "nscannedObjects" : 2,
        "nscanned" : 2,
        "nscannedObjectsAllPlans" : 2,
        "nscannedAllPlans" : 2,
        "scanAndOrder" : false,
        "indexOnly" : false,
        "nYields" : 0,
        "nChunkSkips" : 0,
```

```
        "millis" : 0,
        "indexBounds" : {
                "Category" : [
                        [
                                "Web Mining",
                                "Web Mining"
                        ]
                ]
        },
        "server" : "PUNITP123103L:27017",
        "filterSet" : false
}
>
```

Let us look at the case of covered index. Observe that the "indexOnly" property will be set to true for covered index.

```
> db.books.find({"Category":"Web Mining"},{"Category":1,_id:0}).pretty().hint({"Category":1}).explain();
{
        "cursor" : "BtreeCursor Category_1",
        "isMultiKey" : false,
        "n" : 2,
        "nscannedObjects" : 0,
        "nscanned" : 2,
        "nscannedObjectsAllPlans" : 0,
        "nscannedAllPlans" : 2,
        "scanAndOrder" : false,
        "indexOnly" : true,
        "nYields" : 0,
        "nChunkSkips" : 0,
        "millis" : 0,
        "indexBounds" : {
                "Category" : [
                        [
                                "Web Mining",
                                "Web Mining"
                        ]
                ]
        },
        "server" : "PUNITP123103L:27017",
        "filterSet" : false
}
>
```

In order to have the index cover the query, ensure that only those columns are projected on which the index is built. In the above example, the index is built on the "Category" column, and "Category" is the only column that is projected. Even the identifier (_id) is suppressed.

### 6.5.14 MongoImport

This command used at the command prompt imports CSV (Comma Separated Values) or TSV (Tab Separated Values) files or JSON (Java Script Object Notation) documents into MongoDB.

**Objective:** Given a CSV file "sample.txt" in the D: drive, import the file into the MongoDB collection, "SampleJSON". The collection is in the database "test".

The "sample.txt" file is as follows:

```
_id,FName,LName
1,Samuel,Jones
2,Virat,Kumar
3,Raul,"A Simpson"
4,,"Andrew Simon"
```

**Act:**

At the command prompt, execute the following command:

Mongoimport --db test --collection SampleJSON --type csv --headerline --file d:\sample.txt

On successful execution of the command, the message at the prompt will be as follows:

```
connected to: 127.0.0.1
2015-02-20T21:09:27.301+0530 imported 4 objects
```

**Output:** To confirm the output, log into MongoDB shell and navigate to the "SampleJSON" collection in the "test" database.

The following are the JSON documents in the collection:

```
> db
test
> show collections
Customers
SampleJSON
books
fs.chunks
fs.files
persons
system.indexes
usercounters
users
> db.SampleJSON.find().pretty();
{ "_id" : 1, "FName" : "Samuel", "LName" : "Jones" }
{ "_id" : 2, "FName" : "Virat", "LName" : "Kumar" }
{ "_id" : 3, "FName" : "Raul", "LName" : "A Simpson" }
{ "_id" : 4, "FName" : "", "LName" : "Andrew Simon" }
>
```

### 6.5.15 MongoExport

This command used at the command prompt exports MongoDB JSON documents into CSV (Comma Separated Values) or TSV (Tab Separated Values) files or JSON (Java Script Object Notation) documents.

**Objective:** This command used at the command prompt exports MongoDB JSON documents from "Customers" collection in the "test" database into a CSV file "Output.txt" in the D: drive.

Given below is a snapshot of the JSON documents in the "Customers" collection of the "test" database.

```
> db
test
> show collections
Customers
SampleJSON
books
fs.chunks
fs.files
persons
system.indexes
usercounters
users
> db.Customers.find().pretty();
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5b"),
        "CustID" : "C123",
        "AccBal" : 500,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5c"),
        "CustID" : "C123",
        "AccBal" : 900,
        "AccType" : "S"
}
```

```
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5d"),
        "CustID" : "C111",
        "AccBal" : 1200,
        "AccType" : "S"
}
{
        "_id" : ObjectId("54df6d4f46a31d28183b9a5e"),
        "CustID" : "C123",
        "AccBal" : 1500,
        "AccType" : "C"
}
>
```

**Act:** At the command prompt, execute the following command:

Mongoexport --db test --collection Customers --csv --fieldFile d:\fields.txt --out d:\output.txt

Before executing this command, ensure that you have created a "fields.txt" with a format defined as follows. The "fields.txt" file:

CustID
AccBal
AccType

For the MongoExport command to execute successfully, ensure that the fields are spelt as is in the MongoDB collection. The case also has to be maintained. It is mandatory to ensure that only one field name is placed per line.

On successful execution of the command, the message at the prompt will be as follows:

```
connected to: 127.0.0.1
exported 4 records
```

**Output:** To confirm the output, navigate to the D: drive and check the file "Output.txt".

"Output.txt"
CustID,AccBal,AccType
"C123",500.0,"S"
"C123",900.0,"S"
"C111",1200.0,"S"
"C123",1500.0,"C"

### 6.5.16 Automatic Generation of Unique Numbers for the "_id" Field

**Step 1:** Run the insert() method on a new collection "usercounters". This is to start off with an initial value of 0 for the "seq" field.

```
db.usercounters.insert(
{
_id: "empid",
seq:0
}
)
```

**Step 2:** Create a user-defined function "getnextseq". This method will invoke "findAndModify()" method on the "usercounters" collection. This is to increment the value of seq field by 1 and update the same in "usercounters" collection.

```
function getnextseq(name) {
var ret=db.usercounters.findAndModify(
{
query: {_id:name},
update: {$inc:{seq:1}},
new:true
}
);
return ret.seq;
}
```

**Step 3:** Run the insert() method on the collection where you need to have the "_id" field and get the uniquely generated number. Notice the call to getnextseq() method as value to _id. The return value from the getnextseq() method becomes the value of _id.

```
db.users.insert(
{
_id:getnextseq("empid"),
Name: "sarah jane"
}
)
```